Distributed Training and Serving

Memory Anatomy, Data/Tensor/Pipeline/Sequence Parallelism, ZeRO and FSDP, Multi-GPU Inference

Naeemullah Khan
naeemullah.khan@kaust.edu.sa



جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

October 22, 2026

Table of Contents

1. Motivation: the memory wall — why a 7B model does not fit on an 80 GB GPU
2. Memory anatomy of training: parameters, gradients, optimizer states, activations; recomputation and gradient accumulation
3. Data parallelism: DDP, all-reduce, gradient bucketing, and large-batch learning rates
4. ZeRO and FSDP: sharding the model states across data-parallel ranks
5. Tensor parallelism: Megatron-style intra-layer splits and why they stay inside a node
6. Pipeline parallelism: GPipe, 1F1B, the bubble, and 3D parallelism
7. Sequence and context parallelism: training million-token contexts
8. Frameworks in practice: DeepSpeed, FSDP, Megatron-LM, and Accelerate
9. Precision and communication: bf16/fp8, NCCL collectives, and the interconnect hierarchy
10. Distributed inference: replication, tensor-parallel serving, routing, and failure handling

Table of Contents (cont.)

11. Summary and references

The Model Does Not Fit

- Take the smallest model anyone still calls a "large" language model: **7 billion parameters**. In `bf16` the weights alone are $7 \times 10^9 \times 2 = 14$ GB, which fits comfortably on one GPU.
- **Training** it does not. Adam plus a mixed-precision master copy costs **16 bytes per parameter** of *model state*, before a single activation is stored:

$$7 \times 10^9 \text{ params} \times 16 \frac{\text{bytes}}{\text{param}} = 1.12 \times 10^{11} \text{ bytes} = \mathbf{112\ GB}.$$

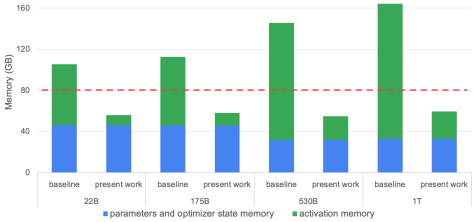
- An NVIDIA H100 SXM has **80 GB** of HBM3. An H200 has 141 GB, a B200 has 192 GB — and the model people actually want to train is 70B or 405B, not 7B.
- So the first question of this lecture is not "how do I go faster?" It is **"how do I run at all?"** Speed is the second question.

Throughout this deck 1 GB $= 10^9$ bytes. Card specifications: NVIDIA H100, H200.

Two Curves That Diverge

- **Model size** has grown by three orders of magnitude in three years; **GPU memory** per device by less than one. Rajbhandari et al. put the gap at "over 1000×" against "5×" (16 GB to 80 GB).
- The scaling laws covered earlier say the loss keeps falling with parameters and tokens. Nothing in them says the result fits in one device.
- Two consequences follow:
  - Training state must be **split** across devices, not replicated.
  - Whatever you split you must later **communicate** — over links running at a quarter of HBM speed inside a node, and under a fiftieth of it between nodes.
- Every technique here answers one question: *which term do I shard, and what does the communication cost?*

Growth figures: Rajbhandari et al., "ZeRO-Infinity," arXiv:2104.07857v1.



Memory per GPU, baseline versus optimised, at four model sizes. The dashed line is the 80 GB of an A100. Korthikanti et al., "Reducing Activation Recomputation in Large Transformer Models," arXiv:2205.05198v1, Figure 1.

Four Axes to Split On

- A training step is a four-dimensional object: a **batch** of examples, each of some **sequence length**, pushed through a stack of **layers**, each holding a **weight matrix**. You can cut along any of the four.

Data parallelism	**Tensor parallelism**	**Pipeline parallelism**	**Sequence parallelism**
cut the *batch*	cut each *weight matrix*	cut the *stack of layers*	cut the *sequence*
replicate the model	inside a layer	into stages	across devices
every rank sees all the weights	all-reduce twice per layer	point-to-point, but has a bubble	attention must see other shards

- ZeRO and FSDP are a fifth idea layered on the first: keep data parallelism's simple programming model, but stop replicating the model state.
- Production pre-training runs use **three or four at once**. We build up to that combination in the pipeline-parallelism section.

Learning Outcomes

By the end of this lecture you should be able to:

- **Compute** the memory footprint of a training run — parameters, gradients, optimizer states, activations — and predict when and why it will run out of memory.
- **Explain and choose between** data, tensor, pipeline, and sequence parallelism, and say what each one costs in communication.
- **Describe** ZeRO stages 1–3 and FSDP sharding, and configure them for a fine-tuning run.
- **Combine** strategies into a 3D-parallel plan for a large pre-training job on a given cluster.
- **Explain** how distributed *inference* differs from distributed *training*: replication versus sharding, routing, and failure handling.
- **Assumed background** (all covered earlier): the transformer block, scaling laws, and single-node inference optimisation — KV caching, continuous batching, quantization.

Where the Bytes Go

- GPU memory during training holds four things. Only the first is the model you think you are training.
  - **Parameters** Ψ — the weights used in the forward pass.
  - **Gradients** — one number per parameter, produced by the backward pass.
  - **Optimizer states** — Adam's two running moments plus the high-precision master copy of the weights.
  - **Activations** — everything the backward pass needs to re-read: layer inputs, attention scores, GeLU inputs, dropout masks.
- The first three scale with **model size** and are called the **model states**. The fourth scales with **batch size** $\times$ **sequence length** and is called the **residual state**.
- The distinction matters because they are attacked by different tools: sharding for model states, recomputation and sequence splitting for activations.

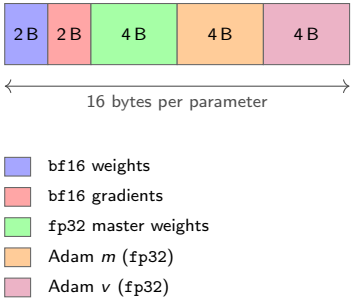
Terminology follows Rajbhandari et al., "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models," arXiv:1910.02054v3, section *Where Did All the Memory Go?*

The 16-Bytes-per-Parameter Rule

Mixed-precision training with Adam, per parameter:

Tensor	dtype	bytes
Weights (forward/backward)	`bf16`	2
Gradients	`bf16`	2
Master copy of weights	`fp32`	4
Adam first moment m	`fp32`	4
Adam second moment v	`fp32`	4
Total model states		**16**

- ZeRO writes this as $2\Psi + 2\Psi + K\Psi = 16\Psi$ bytes with the **optimizer multiplier** $K = 12$.
- **Assumptions:** Adam or AdamW; `fp32` optimizer states; one `fp32` master weight copy; gradients kept in `bf16`; activations counted separately.



The green, orange, and purple blocks — 12 of the 16 bytes — are what ZeRO stage 1 shards first.

Why fp32 Training Is Not Cheaper

- A common misconception: "mixed precision halves training memory." It halves the *activation* and *gradient* memory, but the model-state total is unchanged.
- Pure `fp32` with Adam: weights 4 + gradients 4 + m 4 + v 4 = **16 bytes per parameter**.
- Mixed precision with Adam: $2 + 2 + 4 + 4 + 4 =$ **16 bytes per parameter**.
- Both routes land on the same number, which is why "16Ψ" is the figure to memorise. Mixed precision buys **throughput** (tensor cores) and **activation memory**, not model-state memory.
- What *does* move the number: a different optimizer. SGD with momentum is $2 + 2 + 4 + 4 = 12$ bytes; plain SGD is 8; 8-bit Adam stores m and v in one byte each, giving $2 + 2 + 4 + 1 + 1 = 10$; and full-model `bf16` training without a master copy is $2 + 2 + 4 + 4 = 12$, at some risk to convergence.

Why fp32 Training Is Not Cheaper (cont.)

- Parameter-efficient fine-tuning (LoRA) attacks the same term from the other side: if only 0.1% of parameters carry gradients and optimizer states, 16Ψ collapses to roughly 2Ψ plus a rounding error.

Model States, by Model Size

Model	Parameters Ψ	`bf16` weights	Model states 16Ψ	80 GB GPUs needed
GPT-2 XL	1.5×10^9	3 GB	24 GB	1
Llama-class	7×10^9	14 GB	112 GB	2
Llama-class	70×10^9	140 GB	1.12 TB	14
GPT-3	175×10^9	350 GB	2.8 TB	35
Llama-3 405B	405×10^9	810 GB	6.48 TB	81

- The last column is a **lower bound**: it counts model states only, at 100% packing efficiency, with zero activations, zero fragmentation, and zero communication buffers. Real runs need more.
- Read it as the answer to "how many GPUs before I can even start?" — not "how many GPUs should I use?"
- Note the asymmetry with inference: serving the 70B model needs 140 GB of weights (2 GPUs), training it needs 1.12 TB (14 GPUs). **Training is roughly 8× the memory of inference** for the same model.

Activations: the Term That Depends on Your Batch

- Model states are fixed once you pick the model. **Activations are the term you control** — and the term that explodes with long context.
- For a standard transformer layer in mixed precision, with micro-batch b, sequence length s, hidden size h, and a attention heads, Korthikanti et al. count the stored activations exactly:

$$\text{bytes per layer} \;=\; s\,b\,h\left(34+\frac{5\,a\,s}{h}\right).$$

- The 34 collects the LayerNorm, QKV, linear-layer and GeLU inputs, the stored Q, K and V, and the dropout masks. The $5as/h$ term is the **materialised $s \times s$ attention matrix**: the only piece that is quadratic in sequence length.
- **Worked example**, a 7B-class model with $h = 4096$, $a = 32$, $L = 32$, $s = 2048$, micro-batch $b = 1$:

$$\tfrac{5as}{h} = 80, \quad sbh(34+80) = 8.39 \times 10^6 \cdot 114 \approx 0.96 \text{ GB/layer}, \quad \times 32 \approx \textbf{31 GB}.$$

- Total for one step: $112 + 31 = \textbf{143 GB}$ on a card with 80 GB. Even a micro-batch of *one* does not fit.

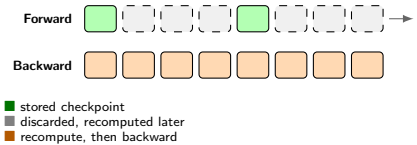
Equation 1 of Korthikanti et al., arXiv:2205.05198v1, Section 4.1.

Two Things That Make Activations Worse, and One That Helps

- **Longer context is superlinear.** Take the same 7B model to $s = 8192$: the quadratic term becomes $5as/h = 320$, so each layer costs $sbh(34 + 320) \approx 11.9\,\text{GB}$ and the stack costs $\approx 380\,\text{GB}$ — a $4\times$ longer sequence for a $12\times$ memory bill.
- **Bigger micro-batches are linear.** Activations scale exactly with b, which is why the first thing that fails when you raise the batch size is memory, not compute.
- **IO-aware attention removes the quadratic term.** FlashAttention never materialises the $s \times s$ matrix, so the $5as/h$ term disappears and the per-layer cost drops to $34sbh$. The stack then costs 9.1 GB instead of 31 GB at $s = 2048$, and 36 GB instead of 380 GB at $s = 8192$.
- FlashAttention and FlashAttention-2 are covered earlier; here we only need the consequence, which is that the *linear* $34sbh$ term is what distributed training has to deal with, and it is still large.

Activation Recomputation (Gradient Checkpointing)

- **The trade.** Do not store the intermediate activations. Store only the *input* to each transformer layer, and recompute the interior with an extra forward pass when the backward pass reaches it.
- **Full recomputation** at layer granularity reduces the whole activation term to $2sbhL$ bytes. For our 7B example that is $2 \cdot 2048 \cdot 4096 \cdot 32 \approx 0.54$ GB, down from 31 GB — a $57\times$ reduction.
- **The price** is one extra forward pass (Chen et al.): about 33% more compute in theory, and Megatron's authors measure **30–40% execution-time overhead** on real training runs.
- **Selective recomputation** recomputes only the cheap-but-huge pieces (the attention softmax and dropout region) and stores the rest. It recovers over 90% of the lost time while keeping most of the memory saving.
- In PyTorch this is `torch.utils.checkpoint`; in Hugging Face, `model.gradient_checkpointing_enable()`.



Store one activation per boundary; rebuild the rest on demand.

Gradient Accumulation: a Bigger Batch Without More Memory

- Large-model training wants a **global batch** of millions of tokens for stable optimisation, but activation memory caps the **micro-batch** you can fit on one device.
- **Gradient accumulation** decouples them: run m micro-batches, sum their gradients into the same buffer, and step the optimizer once.

$$\text{global batch} = \underbrace{b}_{\text{micro-batch}} \times \underbrace{m}_{\text{accumulation steps}} \times \underbrace{d}_{\text{data-parallel ranks}}$$

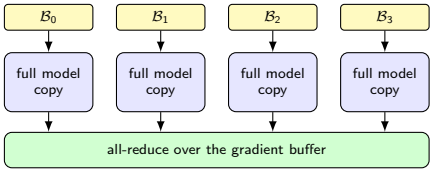
- Memory cost: **zero**. The gradient buffer already exists; only one micro-batch of activations is live at a time.
- Time cost: also close to zero, *provided* you skip the all-reduce on the intermediate micro-batches. PyTorch DDP exposes this as the `no_sync()` context manager; forgetting it multiplies your communication bill by m.
- Watch the loss normalisation: divide by m (or use `mean` reduction consistently), or your effective learning rate silently scales with the accumulation count.
- The same m reappears in pipeline parallelism as the **number of micro-batches**, where it controls the pipeline bubble. It is the single most reused hyperparameter in this lecture.

Data Parallelism: Replicate the Model, Split the Batch

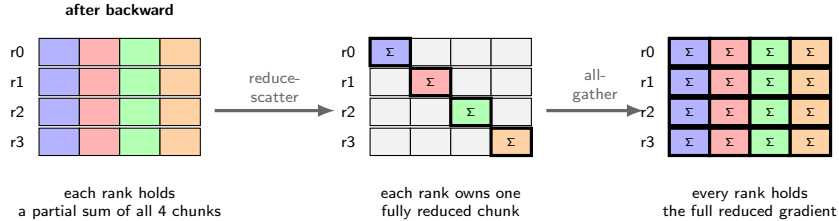
- Give every one of d ranks a **complete copy** of the model and $1/d$ of the global batch. Each rank runs its own forward and backward pass with no communication at all.
- Correctness rests on one fact: the gradient of a sum is the sum of the gradients. If rank r holds micro-batch $\mathcal{B}_r$, then

$$\nabla_\theta \mathcal{L}\Big(\bigcup_r \mathcal{B}_r\Big) \;=\; \frac{1}{d}\sum_{r=1}^{d} \nabla_\theta \mathcal{L}(\mathcal{B}_r).$$

- So the **only** synchronisation needed is one **all-reduce** over the gradient buffer at the end of the backward pass. After it, every rank holds the identical averaged gradient and takes an identical optimizer step, so the replicas never diverge.
- This is why data parallelism is the default: it is one collective per step, it needs no change to the model code, and it scales throughput almost linearly — *as long as the model fits on one device*.



What an All-Reduce Actually Costs



- A ring all-reduce is a **reduce-scatter** followed by an **all-gather**. Each phase moves $\frac{d-1}{d}\Psi$ elements per rank, so the round trip moves $2\frac{d-1}{d}\Psi \approx 2\Psi$ elements — **independent of the number of ranks**.
- In bf16 that is 4Ψ *bytes* per rank per step: 28 GB for a 7B model. Tens of milliseconds inside an NVLink domain; hundreds of milliseconds over a single 400 Gb/s InfiniBand port. It must be hidden, not merely tolerated.

Volume analysis: Rajbhandari et al., arXiv:1910.02054v3, section *Communication Analysis of ZeRO-DP*.

Hiding the Communication: Gradient Bucketing

- The naive implementation waits for the whole backward pass, then all-reduces one enormous buffer. The GPU sits idle for the entire collective.
- **PyTorch DDP** instead groups parameters into **buckets** (25 MB by default) in reverse registration order. As soon as every gradient in a bucket is ready, that bucket's all-reduce is launched on a side stream while the backward pass continues through earlier layers.
- Because the backward pass produces gradients *last layer first*, and the last layers are exactly the ones DDP buckets first, most of the communication overlaps with useful compute.
- Two knobs matter in practice:
  - **Bucket size:** too small and you pay per-collective latency; too large and there is nothing left to overlap with. NCCL needs large messages to reach peak bandwidth.
  - `no_sync()`: skip the all-reduce entirely on gradient-accumulation micro-batches.
- With these in place, Li et al. report **near-linear scalability to 256 GPUs** for DDP.

Hiding the Communication: Gradient Bucketing (cont.)

- A practical corollary: **never build a DP job out of** `DataParallel` (single-process, multi-thread, scatters and gathers every step). Use `DistributedDataParallel` with one process per GPU.

Li et al., "PyTorch Distributed: Experiences on Accelerating Data Parallel Training," VLDB 2020. arXiv:2006.15704v1

Scaling Efficiency: When Data Parallelism Stops Paying

- Per step, the compute per rank scales with the **micro-batch** b, but the communication is a fixed $\approx 4\Psi$ bytes regardless of b. The ratio that decides your efficiency is therefore

$$\frac{\text{communication}}{\text{computation}} \propto \frac{\Psi}{b \cdot s \cdot \Psi} = \frac{1}{b \cdot s}.$$

- Read off the three failure modes:
  - **Small per-device batch** (because activations do not fit) $\Rightarrow$ communication dominates. This is why activation memory and communication efficiency are the same problem.
  - **Slow interconnect** $\Rightarrow$ the same volume takes longer. Ethernet-connected nodes fall off the linear curve far earlier than InfiniBand ones.
  - **Too many ranks** $\Rightarrow$ the global batch grows past the point where extra examples still improve the gradient.

Scaling Efficiency: When Data Parallelism Stops Paying (cont.)

- The last point is not a systems problem but a statistics one. McCandlish et al. formalise it with the **gradient noise scale**: beyond a *critical batch size*, doubling the batch stops halving the number of steps to a given loss, so extra GPUs buy wall-clock time at a worsening compute cost.

McCandlish, Kaplan, Amodei et al., "An Empirical Model of Large-Batch Training," arXiv:1812.06162v1.

Large Batches Change the Optimizer

- Adding data-parallel ranks multiplies the global batch. If you leave the learning rate alone, you take the same-sized steps on a less noisy gradient and converge more slowly per epoch.
- **Linear scaling rule** (Goyal et al.): when the minibatch is multiplied by k, multiply the learning rate by k. This let them train ResNet-50 with a batch of **8192 on 256 GPUs in one hour** with no accuracy loss.
- It only works with a **gradual warmup**: the rule is derived from an approximation that fails in the first few epochs, when weights change fastest. Ramp the learning rate from small to target over a few hundred to a few thousand steps.
- At extreme batch sizes even that breaks down. **LARS** and then **LAMB** rescale the update per layer by the ratio of weight norm to update norm; LAMB trained BERT at batch **32,868**, cutting training from 3 days to **76 minutes**.
- For LLM pre-training the modern practice is a **batch-size warmup** as well: start with a smaller global batch and grow it as the gradient noise scale rises during training.

Large Batches Change the Optimizer (cont.)

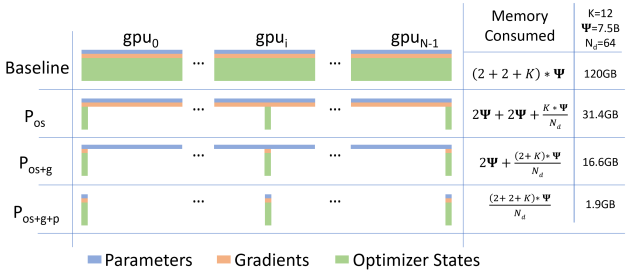
Goyal et al., arXiv:1706.02677v2; You et al., "Large Batch Optimization for Deep Learning: Training BERT in 76 minutes," ICLR 2020, arXiv:1904.00962v5.

The Wall Data Parallelism Cannot Climb

- Data parallelism gives you **throughput**, and nothing else. Every rank still stores the full 16Ψ bytes of model states.
- Adding GPUs therefore does not help you fit a bigger model at all. On an 80 GB card, $80\,\text{GB}/16\,\text{bytes} = 5 \times 10^9$ parameters exactly exhausts memory with *zero* activations; allowing for activations, buffers, and fragmentation the practical ceiling is around **2–3B parameters**. Rajbhandari et al. report the analogous limit as **1.4B** parameters on 32 GB cards.
- Worse, the replication is *pure waste*: d ranks store d identical copies of the optimizer states, and each rank only ever needs $1/d$ of them at the moment of the optimizer step.
- That observation — *the redundancy is unnecessary* — is the whole content of the next section.

ZeRO: Data Parallelism Without the Redundancy

- The insight is almost embarrassingly simple. In DDP, rank r stores all 16Ψ bytes but, at the moment of the optimizer step, only *uses* $1/d$ of the optimizer states to update $1/d$ of the parameters. The rest is idle storage.
- **ZeRO** (Zero Redundancy Optimizer) partitions the model states across the d data-parallel ranks instead of replicating them, and communicates the missing pieces *just in time*. The programming model stays data-parallel; only the storage changes.



Per-device memory of the model states under the three ZeRO-DP stages, for $\Psi = 7.5$B, $N_d = 64$, $K = 12$. Rajbhandari et al., "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models," arXiv:1910.02054v3, Figure 1.

The Three Stages

Stage	What is partitioned	Bytes per rank	Communication
Baseline DDP	nothing	16Ψ	2Ψ (all-reduce)
ZeRO-1 (P_{os})	optimizer states	$4\Psi + 12\Psi/d$	2Ψ (unchanged)
ZeRO-2 (P_{os+g})	+ gradients	$2\Psi + 14\Psi/d$	2Ψ (unchanged)
ZeRO-3 (P_{os+g+p})	+ parameters	$16\Psi/d$	3Ψ $(1.5\times)$

- The reductions in the limit of large d are **4**×, **8**×, and $d\times$. Only stage 3 removes the constant term, which is why only stage 3 lets model size grow with the cluster.
- **Stages 1 and 2 are free.** Standard DDP already implements the all-reduce as a reduce-scatter followed by an all-gather; ZeRO simply keeps the intermediate partitioned result instead of throwing it away. Same 2Ψ of traffic, less memory.
- **Stage 3 costs 50% more traffic.** Parameters must be all-gathered once for the forward pass and once again for the backward pass, on top of the gradient reduce-scatter: $\Psi + \Psi + \Psi = 3\Psi$.
- All communication volumes are counted in *elements*; multiply by 2 for `bf16` bytes.

ZeRO in Numbers

- **A 7B model on one node of 8×80 GB.** Model states are $16\Psi = 112\,\text{GB}$, so DDP is out of the question. Per GPU:

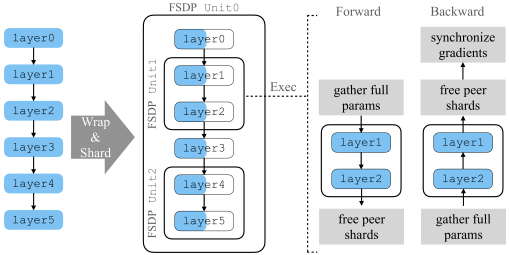
Configuration	Model states / GPU	Free for activations	Verdict
DDP	112 GB	—	out of memory
ZeRO-1, $d = 8$	$4\Psi + 12\Psi/8 = 28 + 10.5 = 38.5$ GB	41.5 GB	fits
ZeRO-2, $d = 8$	$2\Psi + 14\Psi/8 = 14 + 12.25 = 26.3$ GB	53.7 GB	fits, larger batch
ZeRO-3, $d = 8$	$16\Psi/8 = 14$ GB	66 GB	fits, largest batch

- **A 70B model.** Model states are 1.12 TB. On 8 GPUs, ZeRO-3 needs $1120/8 = 140$ GB per GPU — still impossible. On 16 it is 70 GB, which leaves nothing for activations. **32 GPUs** gives 35 GB per GPU and is the first configuration that actually works.
- This arithmetic is the single most useful thing in this lecture: before you launch a job, divide 16Ψ by your GPU count and see whether the answer leaves room to breathe.
- **Offload** extends the ladder downwards. ZeRO-Offload moves optimizer states and the optimizer step to CPU memory and trained **13B parameters on a single GPU**; ZeRO-Infinity adds NVMe and fine-tunes trillion-parameter models on one DGX-2 node. Both trade PCIe bandwidth for capacity, so they are for "it must run" rather than "it must be fast".

Ren et al., arXiv:2101.06840v1; Rajbhandari et al., arXiv:2104.07857v1.

PyTorch FSDP: the Native Implementation

- **Fully Sharded Data Parallel** is PyTorch's in-tree realisation of the ZeRO-3 idea. The model is decomposed into **FSDP units**; each unit i's ψ_i parameters are flattened and sharded across F ranks.
- The runtime loop, per unit: **all-gather** the full parameters just before the unit runs, compute, **free** the peer shards immediately afterwards, and **reduce-scatter** the gradients on the way back.
- Peak *parameter* memory is therefore $\sum_i \psi_i / F + \max_i \psi_i$: the sharded whole, plus *one* materialised unit. That $\max_i \psi_i$ term is the entire reason unit granularity matters.



Zhao et al., "PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel," VLDB 2023. arXiv:2304.11277v2, Figure 1.

Wrapping Policy: the Knob That Decides Everything

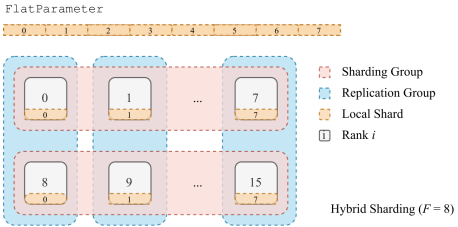
- The **auto-wrap policy** decides where unit boundaries fall, and it sets a direct trade-off:
  - **Fine-grained units** (one per transformer block) $\Rightarrow$ small $\max_i \psi_i$, so low peak memory, but many small collectives.
  - **Coarse units** (one per model) $\Rightarrow$ few large collectives at peak bandwidth, but the whole model must materialise at once, which defeats the point.
- For transformers the right answer is almost always **one FSDP unit per transformer block** — `transformer_auto_wrap_policy` with the block class, or the model's `_no_split_modules` in Hugging Face.
- Two further overlaps are worth knowing because they are on by default and occasionally need turning off:
  - **Forward prefetch:** start the all-gather for unit $i+1$ while unit i computes.
  - **Backward prefetch:** same idea in reverse, which is where most of FSDP's speed comes from.

Wrapping Policy: the Knob That Decides Everything (cont.)

- Zhao et al. measured all-gather time rising sharply once the message drops below 33M elements — concrete evidence for not wrapping every `nn.Linear` separately.
- Combine FSDP with activation checkpointing at the *same* block boundary. The two are orthogonal: FSDP shards the model states, checkpointing shrinks the activations.

Sharding Strategies

- `FULL_SHARD` — parameters, gradients, and optimizer states all sharded. This is ZeRO-3.
- `SHARD_GRAD_OP` — gradients and optimizer states sharded; parameters stay gathered between forward and backward. This is ZeRO-2: one fewer all-gather, more memory.
- `NO_SHARD` — plain replication, i.e. DDP, useful as a baseline inside the same code path.
- `HYBRID_SHARD` — *shard within a node, replicate across nodes*. The expensive all-gathers stay on NVLink; only a gradient all-reduce crosses the slow inter-node fabric.
- `CPUOffload(offload_params=True)` — park parameters and gradients in host memory and run the optimizer step on the CPU. Last resort; PCIe becomes the bottleneck.



Hybrid sharding on 16 GPUs: 2 sharding groups (rows) and 8 replication groups (columns), sharding factor $F = 8$. Zhao et al., arXiv:2304.11277v2, Figure 4.

- Hybrid sharding is the right default the moment a job spans more than one node and the model *does* fit within one node's aggregate memory: it converts the 3Ψ inter-node bill back into 2Ψ.

FSDP1, FSDP2, and Choosing Between DDP and Sharding

- **FSDP1** (`FullyShardedDataParallel`) flattens and concatenates each unit's parameters into a single `FlatParameter`, then chunks it. Efficient, but the flattening hides the original tensors, which complicates frozen parameters, mixed dtypes, and state dicts.
- **FSDP2** (`fully_shard`) shards each parameter individually on dimension 0 as a `DTensor`. Same throughput, but sharded state dicts need no all-gather, frozen parameters just work, and — most importantly — it **composes** with tensor, pipeline, and expert parallelism instead of needing special-case glue. FSDP1 is deprecated; new code should use `fully_shard`.
- **When to prefer sharding over DDP:**
  - Model states do not fit on one device ⇒ you have no choice: ZeRO-3 / `FULL_SHARD`.
  - They fit but leave almost no room ⇒ ZeRO-2 / `SHARD_GRAD_OP` buys batch size at no extra communication.
  - They fit comfortably and the interconnect is slow ⇒ stay on DDP; sharding would add traffic you do not need.

- **The limit of the whole family:** sharding still requires every rank to *materialise* one full layer at a time. When a single layer's weights no longer fit on one GPU, no amount of data-parallel sharding helps — and that is where tensor parallelism starts.

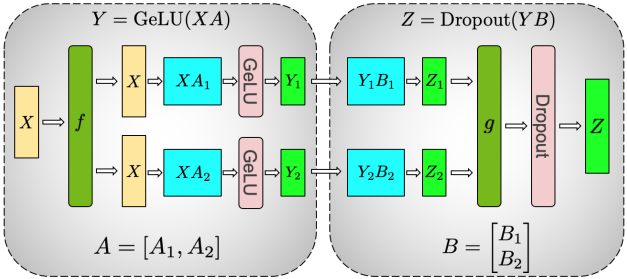
Tensor Parallelism: Split the Matrix, Not the Batch

- Every technique so far kept whole layers on one device. **Tensor parallelism** cuts *inside* a layer, so a single weight matrix lives on t GPUs at once. It is the only tool that helps when one layer alone exceeds one device.
- There are exactly two ways to split a matrix multiply $Y = XA$, and Megatron-LM's contribution is choosing the right one at each position so the two compose with no communication in between.
- **Column-parallel.** Split $A = [A_1, A_2]$ by columns; each rank computes XA_i from a replicated X and holds a slice of the output, $[Y_1, Y_2] = [XA_1,\ XA_2]$. Any *elementwise* nonlinearity then applies independently to each slice — $\mathrm{GeLU}(XA_i)$ is exactly the matching slice of $\mathrm{GeLU}(XA)$ — so nothing needs to be exchanged.
- **Row-parallel.** Split $B = [B_1; B_2]$ by rows and feed it the already-split Y. Each rank now computes a *partial sum* of the full output:

$$Z = YB = Y_1 B_1 + Y_2 B_2 \quad \Longrightarrow \quad \text{one all-reduce to finish.}$$

- Chain them — column-parallel, then row-parallel — and a two-matrix block costs **one** all-reduce instead of two. This is why the split must respect the nonlinearity: splitting A by rows instead would force a synchronisation before the GeLU.

The MLP Block

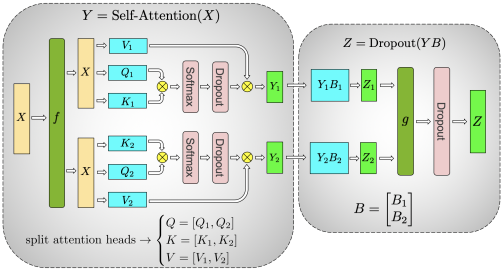


- f and g are **conjugate** operators: f is the identity in the forward pass and an all-reduce in the backward pass; g is an all-reduce in the forward pass and the identity in the backward pass.
- So the whole MLP block costs **one all-reduce forward and one backward**, and the GeLU in the middle is free.
- Note what is *not* split: the input X and output Z are replicated on every rank. Tensor parallelism shards the weights and the intermediate activations, not the block boundaries.

Shoeybi et al., "Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism," arXiv:1909.08053v4, Figure 3(a).

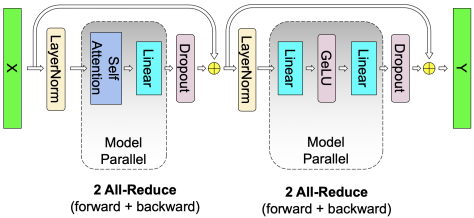
Self-Attention: Sharding Falls Out of the Architecture

- Multi-head attention is *already* a sum over independent heads, so the split needs no new idea: give each rank a disjoint subset of heads.
- The Q, K, V projections are **column-parallel**; the output projection is **row-parallel**. Result: one all-reduce forward, one backward, and the softmax — the expensive part — never crosses a device boundary.
- **Divisibility constraints** bite in practice: t must divide the number of attention heads, the hidden size, and the MLP intermediate size. With grouped-query attention it must also divide the number of *key/value* heads — often only 8, which caps t at 8.
- Embedding and output layers get a **vocabulary-parallel** split with a fused cross-entropy, avoiding an all-gather of the full logits.



Shoeybi et al., arXiv:1909.08053v4, Figure 3(b).

The Communication Bill



Four all-reduces per transformer layer per iteration.

Shoeybi et al., arXiv:1909.08053v4, Figure 4.

- Two all-reduces forward (one per block) and two backward: **four per layer, per iteration**, each on a tensor of $b \cdot s \cdot h$ elements. Full activation recomputation reruns the forward pass, making it six.
- Since an all-reduce moves $\approx 2\times$ the message size per rank, the traffic per layer per iteration is about $8\,b\,s\,h$ elements. For a 7B-class model ($h = 4096$, $L = 32$) at $b = 4$, $s = 2048$ in `bf16`, that is ≈ 17 GB per GPU per step.
- **The volume is not the problem — the placement is.** Data-parallel traffic happens once, at the end of the backward pass, and DDP hides it behind computation. Tensor-parallel traffic sits **on the critical path**: the next operation literally cannot start until the all-reduce lands.
- The compensation is that tensor parallelism shards *activations* too, so it reduces the memory term that ZeRO cannot touch.

Why Tensor Parallelism Stays Inside a Node

- Put the two facts together. Tensor-parallel collectives are (i) frequent — four per layer, so 128 per iteration for a 32-layer model — and (ii) unhideable, because they are synchronisation points in the middle of a layer.
- Now compare the two fabrics they might run over. An H100 SXM has **900 GB/s** of NVLink bandwidth to its neighbours through NVSwitch. A single NDR InfiniBand port gives **50 GB/s**. That is an **18**$\times$ gap on traffic you cannot overlap.
- Hence the standard rule, stated as Takeaway #1 by Narayanan et al.:

  > *"When considering different forms of model parallelism, tensor model parallelism should generally be used up to degree g when using g-GPU servers, and then pipeline model parallelism can be used to scale up to larger models across servers."*

- In practice this means $t \leq 8$ on an 8-GPU HGX node, and the same rule reappears verbatim in inference serving: vLLM's guidance is to set tensor-parallel size to the GPUs per node and pipeline-parallel size to the number of nodes.

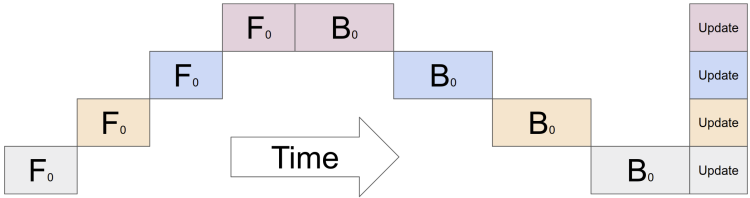
Why Tensor Parallelism Stays Inside a Node (cont.)

- Tensor parallelism also *shrinks* the per-GPU batch, since the batch is replicated across the tensor-parallel group. Pushing t too high starves the GEMMs and drops utilisation even if the bandwidth were free.

Narayanan et al., "Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM," SC 2021. arXiv:2104.04473v5, Section 3.2.

Naive Model Parallelism Wastes Almost Everything

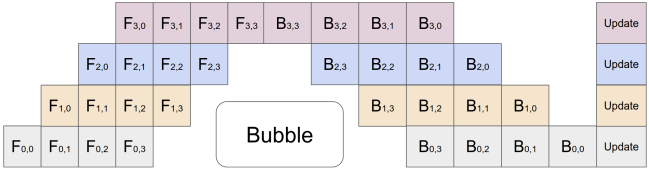
- The obvious way to fit a deep model on p devices is to give each device a contiguous slice of the layers. It works, and it is catastrophically slow.
- Layer $i+1$ cannot start until layer i finishes, so at any instant **exactly one** device is doing work and $p-1$ are idle. Utilisation is $1/p$.



Naive model parallelism across four accelerators: one mini-batch F_0 walks up the stack and back down, and the rest of the cluster waits. Huang et al., "GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism," arXiv:1811.06965v5, Figure 2(b).

- The fix is the same one that made CPU pipelines work: keep several items in flight so the stages overlap.

GPipe: Micro-batching Fills the Pipe



The same four devices after the mini-batch is split into four micro-batches. Huang et al., arXiv:1811.06965v5, Figure 2(c).

- Split the mini-batch into m **micro-batches** and feed them in sequence. While device 1 works on micro-batch 2, device 2 works on micro-batch 1, and so on. Gradients are accumulated and applied once, at the end — so the optimizer semantics are *identical* to single-device training.
- The idle wedges at the start and end are the **pipeline bubble**: $p - 1$ forward passes waiting for the pipe to fill, and $p - 1$ backward passes waiting for it to drain.
- Communication is **point-to-point**, not a collective: each stage sends one activation tensor of size $b \cdot s \cdot h$ to its successor. This is why pipeline parallelism tolerates slow inter-node links — it is the cheapest axis to stretch across a network.

The Bubble, Quantified

- Write p for the number of pipeline stages, m for the number of micro-batches, and t_f, t_b for one micro-batch's forward and backward time. The bubble is $p-1$ forward passes plus $p-1$ backward passes:

$$t_{pb} = (p-1)(t_f + t_b), \qquad t_{id} = m\,(t_f + t_b), \qquad \frac{t_{pb}}{t_{id}} = \boxed{\frac{p-1}{m}}.$$

- GPipe's own paper (writing K for stages and M for micro-batches) states the same quantity as a fraction of *total* time, $\dfrac{p-1}{m+p-1}$, and reports the overhead negligible when $m \geq 4p$. The two measure the same bubble against different baselines, ideal time and total time

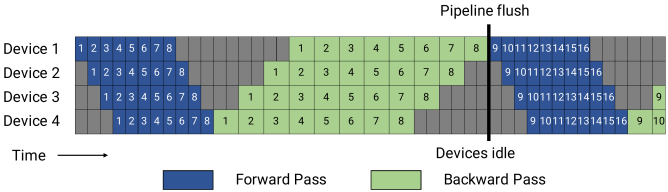
- **Worked example.** $p = 8$ stages, $m = 64$ micro-batches: bubble $= 7/64 \approx 11\%$ of ideal time. Halve the micro-batches to $m = 32$ and it doubles to 22%.

The Bubble, Quantified (cont.)

- So you want $m \gg p$ — and that is exactly where GPipe's memory problem appears. The all-forward-then-all-backward schedule keeps activations for **all** m **micro-batches** alive at once, so shrinking the bubble inflates the activation memory linearly.
- Two constraints pulling in opposite directions through the same variable. Resolving that tension is what the next schedule does.

Narayanan et al., arXiv:2104.04473v5, Section 2.2.1; Huang et al., arXiv:1811.06965v5, Section 2.

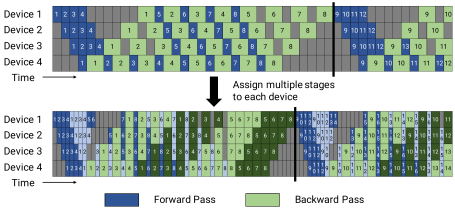
1F1B: Same Bubble, Bounded Memory



The GPipe schedule: all forwards (blue), then all backwards (green), then a flush. Grey is bubble. Narayanan et al., arXiv:2104.04473v5, Figure 3.

- **PipeDream-Flush (1F1B)** keeps the identical bubble but reorders the work: after a short warm-up, every device alternates *one forward, one backward*.
- Because a micro-batch's backward pass now runs almost immediately after its forward pass, the number of micro-batches whose activations must be stashed drops from m to **at most** p.
- When $m \gg p$ — exactly the regime you want for a small bubble — this is a large memory saving for free. 1F1B is the default schedule in Megatron-LM, DeepSpeed, and PyTorch's pipelining API.
- The stages are also unequal in memory: stage 0 holds the most in-flight activations, stage $p-1$ the fewest. Balanced *layer* assignment does not give balanced *memory*.

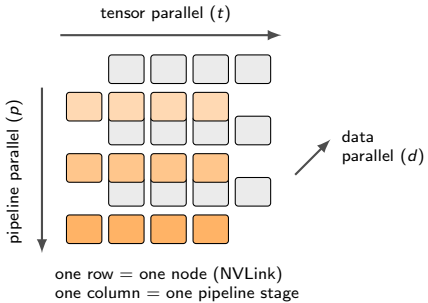
Interleaving: Buying the Bubble Down With Communication



Top: default 1F1B. Bottom: interleaved 1F1B with $v = 2$ chunks per device; the flush happens sooner. Narayanan et al., arXiv:2104.04473v5, Figure 4.

- Give each device v non-contiguous **model chunks** instead of one contiguous block (device 1 gets layers $1, 2$ and $9, 10$; device 2 gets $3, 4$ and $11, 12$; and so on). Each chunk is smaller, so t_f and t_b shrink by v and the bubble becomes $\frac{1}{v} \cdot \frac{p-1}{m}$.
- It is not free: the point-to-point communication volume also increases by v. Interleaving therefore pays off precisely when the pipeline links are fast enough to absorb it — which is why Megatron pairs it with the scatter/gather optimisation over a node's multiple InfiniBand cards.
- Requires m to be an integer multiple of p. Reported gain: over **10%** throughput at comparable memory.

3D Parallelism: Using All Three Axes at Once



- The three axes are orthogonal and multiply: GPUs $= t \times p \times d$.
- They are assigned to the hardware by **communication intensity**: tensor parallelism gets the fastest links (inside a node), pipeline parallelism the slowest (point-to-point across nodes), and data parallelism whatever is left, because DDP traffic can be overlapped.
- Sequence parallelism (next section) adds a fourth, and expert parallelism a fifth — which is why practitioners now say "4D" or "5D" as often as "3D".
- **Mixture-of-Experts routing** is its own parallel axis (each expert lives on its own device, connected by an all-to-all). Routing is covered earlier; here it is enough to know that expert parallelism composes with the three axes above.

A Worked Cluster Plan

- **Given:** 64 nodes, 8 H100 SXM (80 GB) each, NVLink inside a node and InfiniBand between nodes. **Target:** pre-train a 175B-parameter model.
- **Step 1 — tensor parallel.** Set $t = 8$: fill the node, never cross the network with an unhideable all-reduce.
- **Step 2 — pipeline parallel.** Model states are $16\Psi = 2.8\,\text{TB}$. With $t = 8$ alone each GPU would still hold 350 GB. Choose $p = 8$, giving a model-parallel size $M = t \cdot p = 64$ and $2.8\,\text{TB}/64 = 43.8\,\text{GB}$ of model states per GPU.
- **Step 3 — data parallel.** $d = 512/64 = 8$ replicas. Layering a ZeRO-1 distributed optimizer across those 8 ranks shards the 12Ψ optimizer term once more:

$$\frac{4\Psi}{M} + \frac{12\Psi}{M \cdot d} = \frac{700}{64} + \frac{2100}{512} = 10.9 + 4.1 = \mathbf{15.0\ GB\ per\ GPU},$$

  leaving roughly 65 GB per GPU for activations, communication buffers, and headroom.
- **Step 4 — micro-batches.** Pick $m = 64$ per data-parallel replica: bubble $= (p-1)/m = 7/64 \approx 11\%$, or 5.5% with $v = 2$ interleaving. Global batch $= b \cdot m \cdot d$.
- **Sanity check against reality:** Narayanan et al. ran a *1-trillion*-parameter model on 3072 A100s at **502 petaFLOP/s**, 52% of theoretical peak, using exactly this recipe.

Choosing the Configuration

- **Takeaway #1:** use tensor parallelism up to the number of GPUs in a server, then pipeline parallelism across servers.
- **Takeaway #2:** choose the model-parallel size $M = t \cdot p$ so that parameters and intermediate state fit in memory, then use data parallelism to consume the remaining GPUs.
- **Takeaway #3:** the optimal micro-batch size b depends on the model, the pipeline depth p, the data-parallel size d, and the global batch B — there is no universal value; measure it.
- Practical failure modes worth naming out loud:
  - **Unbalanced stages.** The embedding and output layers are much heavier than a transformer block, so an even layer split gives an uneven *time* split, and the slowest stage sets the pace of the whole pipeline.
  - **Too few micro-batches.** A large p with a small global batch produces a bubble that dwarfs the savings.

Choosing the Configuration (cont.)

- **Stragglers.** Every collective is a barrier; one slow GPU or one flapping network link slows all $t \cdot p \cdot d$ of them.

- **Checkpointing at this scale** is itself a distributed problem: state must be saved from every rank, asynchronously, with enough metadata to restore under a *different* parallel layout. PyTorch Distributed Checkpoint and Megatron's converters exist for exactly this.

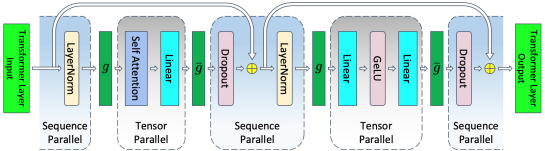
Takeaways paraphrased from Narayanan et al., arXiv:2104.04473v5, Sections 3.2–3.4.

What Tensor Parallelism Leaves Replicated

- Tensor parallelism shards the attention and MLP interiors, but the regions *between* them — LayerNorm, dropout, and the residual adds — are elementwise. They need no weights, so Megatron leaves them replicated on all t ranks.
- Those regions still hold activations. Writing the per-layer activation memory with tensor parallelism of degree t:

$$s\,b\,h\left(10 + \frac{24}{t} + \frac{5as}{ht}\right)$$

  the 10 is the stubborn part: it does not shrink no matter how large t becomes.
- **Sequence parallelism** shards those regions along the **sequence** dimension. Each rank owns s/t positions of the LayerNorm and dropout tensors, which is legal precisely because those operations are independent across positions.



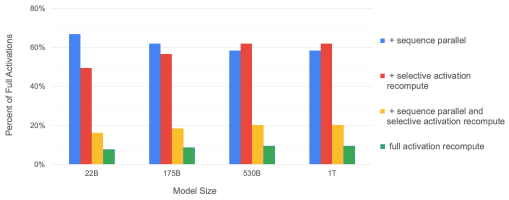
A transformer layer with tensor *and* sequence parallelism. g is an all-gather forward and reduce-scatter backward; $\bar{g}$ is the conjugate. Korthikanti et al., arXiv:2205.05198v1, Figure 5.

Sequence Parallelism Costs Nothing Extra

- The one all-reduce at each tensor-parallel boundary is **replaced** by an all-gather (entering the region) and a reduce-scatter (leaving it).
- Since an all-reduce *is* a reduce-scatter plus an all-gather, the communication volume is **unchanged**. Sequence parallelism is a pure memory win. Per layer it gives

$$\frac{s\,b\,h}{t}\left(34+\frac{5as}{h}\right),$$

  the full activation term divided cleanly by t.
- Add **selective activation recomputation** — recompute only the attention softmax and dropout region, cheap in FLOPs but huge in bytes — and the model's activations collapse to $34sbhL/t$.
- Together: a **5**× memory reduction that recovers over **90%** of the time lost to full recomputation. On a 530B model across 2240 A100s this raised model FLOPs utilisation from 42.1% to **54.2%**.



Activation memory as a percentage of the tensor-parallel baseline. Korthikanti et al., arXiv:2205.05198v1, Figure 7.

Two Different Things Called "Sequence Parallelism"

- This is a genuine naming collision in the literature, and it causes real confusion when reading configuration files.
- **Megatron sequence parallelism** (previous two slides) splits the *non-attention* regions along the sequence, *inside* an existing tensor-parallel group. It does not increase the number of GPUs you can use; it reduces memory at fixed t. Attention still sees the whole sequence.
- **Context parallelism** splits the sequence across a *separate* parallel group and makes attention itself distributed. Each device holds s/c positions of Q, K, and V, and the attention computation must somehow see the other devices' keys and values. This *is* a new axis: it raises the maximum trainable context length in proportion to the device count.
- Two mechanisms dominate:
  - **Ring Attention** (Liu et al.): pass key/value blocks around a ring of devices, overlapping the transfer with blockwise attention compute.

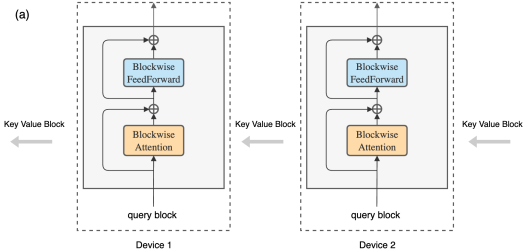
Two Different Things Called "Sequence Parallelism" (cont.)

- **DeepSpeed-Ulysses** (Jacobs et al.): use an all-to-all to switch from a sequence-sharded layout to a head-sharded layout for the attention itself, then switch back. Reported 2.5× speed at 4× the sequence length of the prior state of the art.

- Both are *exact*: they compute the same attention as a single device would, unlike sparse or windowed approximations.

Ring Attention

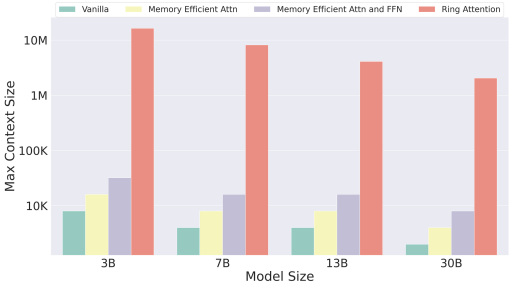
- **The enabling property:** attention between one query block and a set of key/value blocks can be accumulated in *any order*, provided the running softmax statistics are rescaled correctly — the same online-softmax trick FlashAttention uses inside one GPU.
- So arrange the devices in a ring. Each keeps one query block permanently while key/value blocks rotate: a device computes attention against the block it holds while **sending that block onward** and **receiving the next from its predecessor**.
- If blockwise compute takes at least as long as one block transfer, communication is **completely hidden**: zero overhead, no approximation.
- That condition sets a minimum sequence length per device, fixed by the ratio of device FLOPs to interconnect bandwidth. Slower links need bigger blocks.



Key/value blocks traverse a ring of hosts while each host keeps its own query block. Liu, Zaharia & Abbeel, "Ring Attention with Blockwise Transformers for Near-Infinite Context," arXiv:2310.01889v4, Figure 2(a).

How Million-Token Contexts Get Trained

- The headline claim is a scaling statement, not a constant-factor one: Ring Attention supports sequences "up to **device count times longer**" than the memory-efficient baselines.
- The intuition: memory-efficient attention already removed the $O(s^2)$ term on one device, leaving $O(s)$. Sharding the sequence over c devices divides *that* by c, so nothing grows faster than linearly and the ceiling moves with the cluster.
- This is the machinery behind million-token context windows — and the reason they are expensive: the *compute* stays quadratic even when the *memory* does not.
- You will meet it as `context_parallel_size` (Megatron-LM, NeMo) or `sequence_parallel_size` (DeepSpeed), composed with t, p, and d.



Maximum context length in end-to-end training on TPUv4-1024, against vanilla, memory-efficient, and blockwise-parallel transformers. Liu, Zaharia & Abbeel, arXiv:2310.01889v4, Figure 1.

The Landscape

Framework	What it implements	Use it when
PyTorch FSDP	ZeRO-3-style sharding, native and in-tree; `fully_shard` (FSDP2) composes with TP/PP/EP	Fine-tuning and mid-scale pre-training in plain PyTorch; the default choice for most people
DeepSpeed	ZeRO 1/2/3, CPU and NVMe offload, its own pipeline engine, fused kernels	You need offload, or you want the whole run driven by one JSON file
Megatron-LM / Megatron-Core	Tensor, pipeline, sequence, context and expert parallelism; selective recomputation	Pre-training at 100B+, where TP and PP are unavoidable
TorchTitan	Reference PyTorch-native 4D-parallel pre-training stack built on FSDP2	You want Megatron-class parallelism without leaving PyTorch idioms
HF Accelerate, TRL, axolotl	Config-driven wrappers over FSDP/DeepSpeed; the training loop is written for you	Fine-tuning jobs where the science is in the data, not the parallelism

- These are not exclusive. Accelerate *launches* FSDP or DeepSpeed; NeMo wraps Megatron-Core; TorchTitan composes FSDP2 with tensor and pipeline parallelism from `torch.distributed`.
- The real decision is not which library, but **which parallelism strategy** — and that is decided by the arithmetic in the memory-anatomy section, not by the framework.

Config Anatomy: DeepSpeed

```
{
  "train_micro_batch_size_per_gpu": 4,
  "gradient_accumulation_steps": 8,
  "gradient_clipping": 1.0,
  "bf16": { "enabled": true },
  "zero_optimization": {
    "stage": 3,
    "overlap_comm": true,
    "contiguous_gradients": true,
    "offload_optimizer": {
      "device": "cpu",
      "pin_memory": true
    },
    "stage3_gather_16bit_weights_on_model_save": true
  },
  "activation_checkpointing": {
    "partition_activations": true,
    "cpu_checkpointing": false
  }
}
```

```
deepspeed --num_gpus 8 train.py --deepspeed ds_config.json
```

- `stage` is the only line that changes the memory formula: 1, 2, or 3 selects $4\Psi + 12\Psi/d$, $2\Psi + 14\Psi/d$, or $16\Psi/d$.
- `offload_optimizer` moves the 12Ψ term to host memory and runs the Adam step on the CPU. It is the difference between "does not fit" and "fits, slowly".
- The global batch is implied, not stated: $4 \times 8 \times 8 = 256$ sequences on 8 GPUs. Getting this wrong is the most common cause of "why does my loss curve look different from the paper?"
- The mouthful `stage3_gather_16bit_weights_on_model_save` matters more than it looks: without it your checkpoint is a pile of shards rather than a loadable model.

Config Anatomy: FSDP2 and Accelerate

PyTorch FSDP2, explicit:

```
from torch.distributed.device_mesh import init_device_mesh
from torch.distributed.fsdp import fully_shard,
      MixedPrecisionPolicy

mesh = init_device_mesh("cuda", (world_size,))
mp = MixedPrecisionPolicy(param_dtype=torch.bfloat16,
                          reduce_dtype=torch.float32)

# One FSDP unit per transformer block, then the root.
for block in model.model.layers:
    fully_shard(block, mesh=mesh, mp_policy=mp)
fully_shard(model, mesh=mesh, mp_policy=mp)
```

Accelerate, declarative:

```
distributed_type: FSDP
fsdp_config:
  fsdp_version: 2
  fsdp_reshard_after_forward: true      # was FULL_SHARD
  fsdp_auto_wrap_policy: TRANSFORMER_BASED_WRAP
  fsdp_transformer_layer_cls_to_wrap: LlamaDecoderLayer
  fsdp_state_dict_type: SHARDED_STATE_DICT
  fsdp_cpu_ram_efficient_loading: true
  fsdp_offload_params: false
mixed_precision: bf16
```

- The wrapping loop is the whole design: **one unit per transformer block**. Wrap only the root and you gain nothing; wrap every linear layer and you drown in small collectives.
- `reduce_dtype=float32` keeps the gradient reduction in full precision even though the parameters are `bf16`. This is cheap insurance against summation error across hundreds of ranks.
- `reshard_after_forward=False` is FSDP2's spelling of `SHARD_GRAD_OP` (ZeRO-2): keep the parameters gathered between forward and backward, trading memory for one fewer all-gather.
- Launching is the same either way, via `torchrun`:

```
torchrun --nnodes 4 --nproc_per_node 8 \
         --rdzv_backend c10d train.py
```

Choosing by Scale

Job	Hardware	Recipe
7B, LoRA / QLoRA fine-tune	1 GPU, 24–80 GB	No distribution needed. Only the adapters carry gradients and optimizer states, so 16Ψ collapses to roughly 2Ψ.
7B, full fine-tune	8×80 GB, 1 node	FSDP `FULL_SHARD` (or ZeRO-3) + activation checkpointing + `bf16`. Model states $112/8 = 14$ GB per GPU.
70B, full fine-tune	32×80 GB, 4 nodes	FSDP `HYBRID_SHARD` + checkpointing. Model states $1120/32 = 35$ GB per GPU. On 8 or 16 GPUs the arithmetic does not close.
70B, long context ($\geq$ 32k)	32–64 GPUs	Add context parallelism; activations, not weights, are now the binding constraint.
175B+, pre-train from scratch	100s–1000s of GPUs	Megatron-style $t \times p \times d$ + sequence parallelism + selective recomputation + a ZeRO-1 distributed optimizer.

- **The escalation order is fixed, and it is worth memorising:** `bf16` $\rightarrow$ activation checkpointing $\rightarrow$ gradient accumulation $\rightarrow$ ZeRO-2 $\rightarrow$ ZeRO-3/FSDP $\rightarrow$ offload $\rightarrow$ tensor parallel $\rightarrow$ pipeline parallel. Each step costs more complexity than the last; stop as soon as the job fits.
- Do the division *before* you request the allocation. Most failed jobs at this scale are arithmetic errors, not bugs.

Numeric Formats: Range Versus Resolution

Format	Sign	Exponent	Mantissa	Bytes	Character
`fp32`	1	8	23	4	the reference
`tf32`	1	8	10	4*	`fp32` range, `fp16` precision; a tensor-core input format
`fp16`	1	5	10	2	more precision, much less range
`bf16`	1	8	7	2	`fp32`'s exponent, so `fp32`'s range
`fp8 E4M3`	1	4	3	1	forward pass: weights and activations
`fp8 E5M2`	1	5	2	1	backward pass: gradients need range

*`tf32` occupies 32 bits in memory; only the multiplier inputs are truncated.

- The exponent field is what matters for training stability. `fp16` has 5 exponent bits, so its smallest normal number is about 6×10^{-5}: deep-network gradients routinely fall below that and **flush to zero**.
- `bf16` keeps all 8 exponent bits and simply throws away mantissa precision. It has the same range as `fp32` — which is why it needs no rescaling tricks, and why it displaced `fp16` as the default for LLM training.
- Whatever the compute format, the **master weights and the optimizer moments stay in** `fp32`: they accumulate tiny updates over hundreds of thousands of steps, and that is precisely where precision loss compounds.

Loss Scaling, and Why You May Never Need It

- With `fp16`, the gradient distribution sits mostly in the underflow region of the format. Micikevicius et al.'s fix is disarmingly simple: multiply the **loss** by a constant S before the backward pass.
- By the chain rule every gradient is scaled by the same S, shifting the whole distribution into the representable range. Divide the gradients by S before the optimizer step and the update is mathematically unchanged.
- **Dynamic loss scaling** tunes S automatically: raise it every few hundred healthy steps, and on any step that produces an `inf` or `NaN`, halve it and *skip* that step. This is what `torch.amp.GradScaler` does.
- The same paper introduced the other half of the recipe: **keep an** `fp32` **master copy of the weights** that accumulates the updates. Those four bytes are a quarter of the sixteen-byte budget.

Loss Scaling, and Why You May Never Need It (cont.)

- **With `bf16` none of this is needed.** Its range matches `fp32`, so gradients do not underflow, no scaler is required, and no steps are skipped. On any Ampere-or-later GPU, use `bf16`. Know loss scaling because you will read it in every paper written before 2021 and meet it in older codebases.

Micikevicius et al., "Mixed Precision Training," ICLR 2018. arXiv:1710.03740v3

FP8: Halving the Bytes Again

- Hopper and Blackwell tensor cores execute matrix multiplies in **8-bit floating point**, roughly doubling throughput over `bf16` and halving activation and communication bytes.
- Two encodings, because one cannot do both jobs: **E4M3** (max magnitude 448) for the forward pass, where precision matters more than range, and **E5M2** (max magnitude 57344) for gradients, which span a far wider range. E5M2 follows IEEE 754; E4M3 buys extra range by dropping infinities and keeping a single NaN pattern.
- `fp8` is a *compute* format, not a storage format for the whole model. In every working recipe, the master weights and optimizer states stay `fp32`, and the numerically delicate operators — embeddings, the output head, normalisation, softmax, and any router — stay in higher precision.
- Making it stable at scale needs **fine-grained scaling**. DeepSeek-V3 quantises in 1×128 tiles or 128×128 blocks rather than per tensor, and promotes partial sums to CUDA cores for accumulation — the first large public run to use E4M3 in the *backward* pass as well as the forward.

FP8: Halving the Bytes Again (cont.)

- Practically: enable `fp8` through Transformer Engine or TorchAO, verify against a `bf16` run on a short schedule, and keep the loss curves side by side. The failure mode is a slow divergence, not an immediate crash.

Micikevicius et al., "FP8 Formats for Deep Learning," arXiv:2209.05433v2; DeepSeek-AI, "DeepSeek-V3 Technical Report," arXiv:2412.19437v2 (FP8 training framework).

NCCL Collectives at a Glance

Collective	Volume per rank	Where it appears in this lecture
`AllReduce`	$2\frac{d-1}{d}\Psi$	DDP gradient sync; tensor-parallel block outputs
`ReduceScatter`	$\frac{d-1}{d}\Psi$	ZeRO-2/3 and FSDP gradients; sequence-parallel exits
`AllGather`	$\frac{d-1}{d}\Psi$	ZeRO-3 and FSDP parameters; sequence-parallel entries
`Broadcast`	Ψ	synchronising initial weights from rank 0
`AllToAll`	varies	MoE expert routing; DeepSpeed-Ulysses
`Send/Recv`	one activation tensor	pipeline stage boundaries

- **An all-reduce is exactly a reduce-scatter followed by an all-gather.** Almost every design in this lecture is a decision about *where to cut that identity in half* and what to keep in between.
- Every collective is a **barrier**. All ranks in the group must arrive, so the slowest determines the pace and a single hung rank stalls the job. Distinct process groups (one per parallel axis) let independent collectives proceed concurrently.
- Bandwidth is only reached at **large message sizes**, which is why bucketing (DDP) and unit granularity (FSDP) exist at all.
- Useful diagnostics: `nccl-tests` for a bandwidth floor, `NCCL_DEBUG=INFO` to confirm the topology NCCL actually chose, and the PyTorch profiler to see whether communication really overlaps compute.

The Interconnect Hierarchy Decides the Strategy

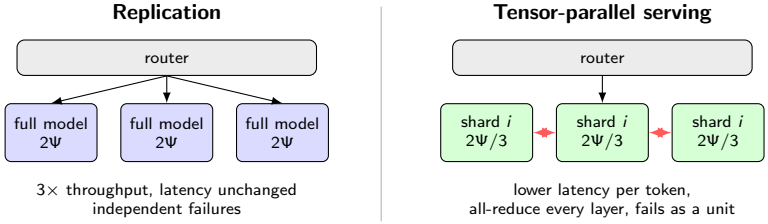
Link	Scope	Bandwidth	Carries
HBM3 (H100 SXM)	on package	3.35 TB/s	everything the GPU touches
NVLink 4 (H100 SXM)	within a node	900 GB/s	tensor parallelism, sequence parallelism
NVLink 5 (B200)	within a node	1.8 TB/s	as above, with more headroom
PCIe Gen5 $\times 16$	host–device	128 GB/s	CPU/NVMe offload; GPUs without NVLink
InfiniBand NDR	node–node	50 GB/s	pipeline stages, data-parallel all-reduce
100 GbE	node–node	12.5 GB/s	the same, on a commodity cluster

- Read the table as a set of cliffs. HBM to NVLink is roughly $4\times$; NVLink to InfiniBand is roughly $18\times$; InfiniBand to Ethernet another $4\times$. **Every parallelism decision in this lecture is an attempt to keep the chattiest traffic above a cliff.**
- That is the single rule behind all of it: tensor and sequence parallelism above the NVLink cliff; pipeline parallelism (point-to-point, low volume) below it; data parallelism below it too, because its traffic can be overlapped with the backward pass.
- It also explains FSDP's `HYBRID_SHARD`: shard within the node where all-gathers are cheap, replicate across nodes where they are not.
- **On a cluster without NVLink or InfiniBand**, the honest advice is different: keep the model on one GPU, use gradient accumulation to reach the global batch you want, and reach for LoRA before reaching for tensor parallelism.

Serving Is Not Training

- Three of the four memory terms simply vanish at inference: no gradients, no optimizer moments, no master weights. 16Ψ **becomes** 2Ψ**.**
- But the workload inverts in three ways that change every design decision:
  - **One long job becomes many short ones.** Training is a single synchronous job over all ranks; serving is millions of independent requests with individual latency targets.
  - **A new state appears.** The KV cache grows with every generated token and is *per request*, not per model.
  - **The bottleneck moves.** Decode reads the whole weight matrix to produce one token, so it is memory-bandwidth-bound rather than compute-bound.
- **Scope.** The single-node story (KV caching, continuous batching, PagedAttention, speculative decoding, quantization, distillation, pruning) was covered earlier. This section covers only what changes when the deployment spans *several GPUs or several nodes*.
- The multi-GPU question has exactly two answers, and real deployments use both at once: **replicate** the model, or **shard** it.

Replication Versus Tensor-Parallel Serving



- **Replicate when the model fits.** N replicas give $N\times$ throughput at unchanged per-request latency, and each replica fails independently. This is the cheapest and most robust axis — always exhaust it first.
- **Shard when it does not, or when latency matters.** Because decode is bandwidth-bound, splitting the weights across t GPUs also splits the bytes each GPU must read per token, so tensor parallelism genuinely reduces time-per-output-token — not merely memory.
- The cost is two all-reduces per layer *per generated token*, on the critical path. The same NVLink rule applies: **keep the tensor-parallel group inside one node**. vLLM's guidance is TP = GPUs per node, PP = number of nodes.

The KV Cache Is the Real Constraint

- Weights are a fixed cost paid once. The KV cache is a *per-request* cost that grows with every token, and it is what actually caps concurrency.
- Per token, per layer, the cache holds a key and a value for each KV head:

$$\text{bytes} = 2 \times n_{kv} \times d_{\text{head}} \times \text{bytes per element}.$$

- **Worked example — a 70B model with grouped-query attention** ($L = 80$ layers, $n_{kv} = 8$, $d_{\text{head}} = 128$, `bf16`), served with $t = 4$ on 4×80 GB:
  - Per token per layer: $2 \times 8 \times 128 \times 2 = 4096$ bytes.
  - Per token, all layers: $4096 \times 80 = 327{,}680$ bytes ≈ 0.33 MB.
  - An 8192-token conversation: $0.33\,\text{MB} \times 8192 \approx$ **2.7 GB** — for *one* request.
  - Budget: $4 \times 80 = 320$ GB total, minus 140 GB of weights and roughly 20 GB of workspace leaves ≈ 160 GB, so about **60 concurrent 8k-token sessions**.
- Three consequences that are easy to miss:

The KV Cache Is the Real Constraint (cont.)

- Tensor parallelism shards the *KV heads* too, so it raises the aggregate cache budget as well as splitting the weights.
- Without GQA this model would need 64 KV heads instead of 8 — 8× the cache, or roughly 7 concurrent sessions instead of 60.
- Replicas do *not* share a KV cache. Adding replicas multiplies capacity but never pools it.

Routing: the Scheduler Above the Replicas

- With N replicas you need a router, and the naive choices are all wrong. **Round-robin fails** because request cost varies by orders of magnitude: a 200-token completion and a 100k-token document summary are not interchangeable units of work.
- **Route on state, not on count.** The useful signals are queue depth, the number of active sequences, and — most importantly — how much KV cache each replica has left. A replica with free HBM can absorb a new request; one at 95% cache occupancy will start preempting.
- **Prefix-cache-aware routing** is the highest-value addition in practice. Requests that share a long system prompt or a document prefix should land on the *same* replica, where that prefix is already in the KV cache. Hashing on the prompt prefix can turn a full prefill into a cache hit.

Routing: the Scheduler Above the Replicas (cont.)

- **Prefill/decode disaggregation** takes this one step further. The two phases have opposite bottlenecks — prefill is compute-bound, decode is memory-bandwidth-bound — and colocating them makes each interfere with the other. DistServe assigns them to *different* GPU pools with independently chosen parallelism, then ships the KV cache between them, reporting **7.4× more requests or 12.6× tighter latency targets**; Splitwise makes the same split across differently specified hardware.

- Continuous batching and PagedAttention (covered earlier) operate *within* a replica; routing and disaggregation operate *across* them. Both are needed, and neither substitutes for the other.

Zhong et al., "DistServe," OSDI 2024, arXiv:2401.09670v3; Patel et al., "Splitwise," ISCA 2024, arXiv:2311.18677v2.

Failure Handling: Two Very Different Problems

Training: all-or-nothing

- Every collective is a barrier, so **one dead rank kills the step** — and with it the whole job.
- Recovery means restarting from the last checkpoint, so checkpoint frequency directly bounds the work you can lose.
- This is not a rare-event concern at scale. Meta's Llama 3 405B run on 16,384 H100s logged **419 unexpected interruptions in 54 days** — roughly one every three hours — with about 78% traced to hardware, GPUs alone accounting for 58.7%.
- Mitigations: asynchronous sharded checkpointing, elastic launchers with automatic restarts, health checks that eject a bad node before it corrupts a step, and hot spare nodes.

Serving: degrade, do not die

- The unit of failure is a **replica**, and a tensor-parallel group is one replica: lose a single GPU and the whole group goes down.
- So a wider tensor-parallel degree is a *worse* availability story. Prefer the smallest t that meets your latency target, and get throughput from replicas.
- In-flight requests on a failed replica are lost. Their KV cache is not replicated anywhere — and replicating it would cost more than re-running the prefill.
- Mitigations: health probes, drain-then-restart on deploys, request-level retries with idempotent semantics, and capacity headroom so the survivors absorb the load.

- The asymmetry is worth stating plainly: **training optimises for progress, serving optimises for availability.** That is why training tolerates deep parallelism and serving does not.

Llama Team, AI @ Meta, "The Llama 3 Herd of Models," arXiv:2407.21783v3, Section 3.3.4 (Reliability and Operational Challenges).

Summary

- **Memory anatomy first.** Mixed-precision Adam costs **16 bytes per parameter** $(2+2+4+4+4)$, so a 7B model needs 112 GB before a single activation. Activations add $sbh(34+5as/h)$ per layer. Every strategy below shards one of those terms.
- **Data parallelism** shards nothing: it replicates the model and splits the batch, buying throughput at one all-reduce (2Ψ) per step, hidden behind the backward pass by bucketing. It cannot make a model fit.
- **ZeRO / FSDP** shards the model states across data-parallel ranks — $4\Psi + 12\Psi/d$, then $2\Psi + 14\Psi/d$, then $16\Psi/d$. Stages 1 and 2 are free; stage 3 costs $1.5\times$ the traffic. `HYBRID_SHARD` keeps the expensive collectives inside a node.
- **Tensor parallelism** splits weight matrices inside a layer (column-parallel, then row-parallel). Four all-reduces per layer per iteration, on the critical path — so it stays inside an NVLink domain, $t \leq 8$.
- **Pipeline parallelism** splits the layer stack and pays a bubble of $(p-1)/m$, or $\frac{1}{v} \cdot \frac{p-1}{m}$ with interleaving. 1F1B keeps that bubble while bounding stashed activations to p rather than m. Its traffic is point-to-point, so this is the axis that crosses nodes.
- **Sequence parallelism** shards the LayerNorm/dropout regions at no extra communication ($5\times$ less activation memory with selective recomputation). **Context parallelism** (Ring Attention, Ulysses) is a different thing with a similar name: it shards attention itself, scaling context length with device count.

Summary (cont.)

- **3D parallelism** composes them by communication intensity: GPUs $= t \times p \times d$, tensor inside the node, pipeline across nodes, data parallelism wherever it overlaps.
- **Serving inverts the problem.** 16Ψ collapses to 2Ψ, the KV cache becomes binding, and availability beats depth: replicate first, shard only for capacity or latency, route on cache state.

The One Slide to Keep

- **Before you request an allocation, do this arithmetic:**
  1. Model states: 16Ψ bytes. Divide by the GPU count. Does it leave headroom?
  2. Activations: $\approx 34sbhL$ bytes per micro-batch with IO-aware attention, or $2sbhL$ with full recomputation. Add it.
  3. If the total still exceeds device memory, walk the escalation ladder — `bf16`, checkpointing, accumulation, ZeRO-2, ZeRO-3, offload, tensor parallel, pipeline parallel — and stop at the first rung that fits.
  4. Then, and only then, ask about speed.
- **And keep the cliffs in mind:** HBM 3.35 TB/s $\rightarrow$ NVLink 900 GB/s $\rightarrow$ InfiniBand 50 GB/s $\rightarrow$ Ethernet 12.5 GB/s. Unhideable traffic belongs above a cliff; overlappable traffic can live below one.

References

1. Rajbhandari, Rasley, Ruwase, He, "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models," SC 2020. arXiv:1910.02054v3
2. Shoeybi, Patwary, Puri, LeGresley, Casper, Catanzaro, "Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism," 2019. arXiv:1909.08053v4
3. Huang et al., "GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism," NeurIPS 2019. arXiv:1811.06965v5
4. Narayanan et al., "Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM," SC 2021. arXiv:2104.04473v5
5. Zhao et al., "PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel," VLDB 2023. arXiv:2304.11277v2
6. Korthikanti, Casper, Lym, McAfee, Andersch, Shoeybi, Catanzaro, "Reducing Activation Recomputation in Large Transformer Models," MLSys 2023. arXiv:2205.05198v1
7. Liu, Zaharia, Abbeel, "Ring Attention with Blockwise Transformers for Near-Infinite Context," ICLR 2024. arXiv:2310.01889v4
8. Jacobs et al., "DeepSpeed Ulysses: System Optimizations for Enabling Training of Extreme Long Sequence Transformer Models," 2023. arXiv:2309.14509v2

References (cont.)

9. Li et al., "PyTorch Distributed: Experiences on Accelerating Data Parallel Training," VLDB 2020. arXiv:2006.15704v1
10. Ren et al., "ZeRO-Offload: Democratizing Billion-Scale Model Training," USENIX ATC 2021. arXiv:2101.06840v1
11. Rajbhandari, Ruwase, Rasley, Smith, He, "ZeRO-Infinity: Breaking the GPU Memory Wall for Extreme Scale Deep Learning," SC 2021. arXiv:2104.07857v1
12. Micikevicius et al., "Mixed Precision Training," ICLR 2018. arXiv:1710.03740v3
13. Micikevicius et al., "FP8 Formats for Deep Learning," 2022. arXiv:2209.05433v2
14. Chen, Xu, Zhang, Guestrin, "Training Deep Nets with Sublinear Memory Cost," 2016. arXiv:1604.06174v2 (activation recomputation)
15. Goyal et al., "Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour," 2017. arXiv:1706.02677v2 (linear scaling rule, gradual warmup)
16. You et al., "Large Batch Optimization for Deep Learning: Training BERT in 76 minutes," ICLR 2020. arXiv:1904.00962v5 (LARS, LAMB)
17. McCandlish, Kaplan, Amodei, OpenAI Dota Team, "An Empirical Model of Large-Batch Training," 2018. arXiv:1812.06162v1 (gradient noise scale, critical batch size)
18. Dettmers, Lewis, Shleifer, Zettlemoyer, "8-bit Optimizers via Block-wise Quantization," ICLR 2022. arXiv:2110.02861v2

References (cont.)

19. Narayanan et al., "Memory-Efficient Pipeline-Parallel DNN Training," ICML 2021. arXiv:2006.09503v3 (PipeDream-2BW and the 1F1B flush schedule)
20. DeepSeek-AI, "DeepSeek-V3 Technical Report," 2024. arXiv:2412.19437v2 (FP8 training at scale)
21. Llama Team, AI @ Meta, "The Llama 3 Herd of Models," 2024. arXiv:2407.21783v3 (4D parallelism and cluster reliability at 16k GPUs)
22. Zhong et al., "DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving," OSDI 2024. arXiv:2401.09670v3
23. Patel et al., "Splitwise: Efficient generative LLM inference using phase splitting," ISCA 2024. arXiv:2311.18677v2
24. Kwon et al., "Efficient Memory Management for Large Language Model Serving with PagedAttention," SOSP 2023. arXiv:2309.06180v1 (vLLM)
25. Liang et al., "TorchTitan: One-stop PyTorch native solution for production ready LLM pre-training," 2024. arXiv:2410.06511v3
26. PyTorch documentation, *FullyShardedDataParallel* and *fully_shard* (FSDP2). FSDP1, FSDP2
27. Hugging Face Accelerate documentation, *Fully Sharded Data Parallel* and *FSDP1 vs FSDP2*. Guide, Comparison
28. DeepSpeed documentation, *ZeRO configuration*. Configuration reference

References (cont.)

29. vLLM documentation, *Parallelism and Scaling*. Guide
30. NVIDIA, *H100 Tensor Core GPU* and *H200* product pages (HBM capacity, NVLink and PCIe bandwidths). H100, H200
31. NVIDIA, *ConnectX-7 NDR 400G InfiniBand Adapter Card* datasheet. PDF